

Validation of agile methodology as ideal software development process using Fuzzy-TOPSIS method

Nikhil Govil^{*}, Ashish Sharma

Department of CEA, IET, GLA University, 17 km Stone, NH-2, Mathura-Delhi Road, P.O. Chaumuhan, Mathura, 281 406, (U.P.), INDIA

ARTICLE INFO

Keywords:

Software Development Process
Decision support system
Fuzzy logic
Agile Software Development
Fuzzy TOPSIS
Multi-Criteria Decision Making

ABSTRACT

Agile methodologies have been an emerging choice of software professionals for the past decade and a half. However, apart from this, some other SDLC models are also available for selection in front of software developers to develop any software. Usually, project managers select any of these models to develop software through their past experiences. There is no logical basis for this selection to be completely correct, as a result of which there is always a risk of software failure or over budget if an inappropriate model has opted. Keeping this problem of software industries in mind, an ideal SDLC model has been identified mathematically in this article. In this article, we applied the Fuzzy TOPSIS method that validates Agile software development as an ideal choice. We have taken a total of six software development processes that are being applied globally. Feedback from five experienced decision-makers has been taken in the form of linguistic terms and further converted into fuzzy values to perform the computation of the closeness coefficient rank of each experimented alternative software development process.

1. Introduction

Decision support systems (DSS) are playing a vital role in different sectors of organizations nowadays. These sectors include corporate houses for efficient business strategies, banking, agriculture, science & technology, and even the entertainment sector as well. Due to the high and increasing demand for the decision-making method, this field is continuously attracting the researchers. The research community is trying to implement a more precise mechanism so that a decision support system can provide results that are more competent as well as more domains that are new. A recent survey [1] shows that different Multi-criteria decision making (MCDM) methods like AHP (Analytic Hierarchy Process), Fuzzy AHP, Fuzzy TOPSIS (Technique for Order Preference by Similarities to Ideal Solution), VIKOR (VIekriterijumsko KOmpromisno Rangiranje), etc. are frequently adopted to increase the efficiency of the decision making process.

Software industry also requires such an efficient DSS at its discrete phases to ensure the overall success of the software system. The success of any software product is utterly dependent on many circumstances. These circumstances may be associated with many factors majorly product attributes, computer or h/w attributes, project attribute and personal attribute [2]. These elements may have a direct impact on

capabilities & experience of professionals, overall development environment, planning strategies, and understanding of requirements, resource identification & utilization timely. To ensure the overall success of software products, professionals are required to opt for the appropriate software development life cycle model among a pool of wide variety of SDLC models. Today it is a big challenge to identify and select the most suitable SDLC model for the project. MCDM methods can be applied to solve this question. In this article, we applied the Fuzzy TOPSIS method that validates Agile software development as an ideal choice among the experimented alternatives.

Software development is a challenging task in itself as every software usually has different constraints, environment, team size, etc. It is a very common phenomenon these days to change the requirement by the client(s) at any stage of software development. Considering this challenge posed by the client(s), the selection of an accurate SDLC model becomes very important as only a competent SDLC model can welcome changes in requirements and avoid project failures or delays. The project manager and other team members have to opt for the most suitable model for developing any software among the variety of various SDLC models. This selection of appropriate SDLC model is not so easy task & it must be validated through a mathematical model to increase the confidence level of development teams. Through this article, we validated

^{*} Corresponding author.

E-mail address: nikhil.govil@gla.ac.in (N. Govil).

<https://doi.org/10.1016/j.advengsoft.2022.103125>

Received 31 January 2022; Received in revised form 12 March 2022; Accepted 30 March 2022

Available online 12 April 2022

0965-9978/© 2022 Elsevier Ltd. All rights reserved.

such a mathematical model that can yield the ideal choice among the experimented alternatives.

The prime objective of our research is to identify the ranking of all alternative and experimented software development processes. Therefore, the development team members along with other stakeholders can select the pertinent model for software development. For our research work, we considered the fuzzy TOPSIS method for rank identification. One of the main challenges during our research work was to determine the criteria through which a particular software system can be affected a lot. We further gathered the corresponding weights in linguistic terms for these criteria from five different decision-makers who are having sound experience in software development processes. The leading motivation behind our research work is to adopt a multi-criteria decision-making method that can compute the ranking of all alternative and experimented models. Hence, the appropriate development process can be opted for a specific software project.

This research article is structured as follows: Section 2 gives a brief overview of the literature and similar works from researchers trying to identify the ideal choice among alternative models. Section 3 provides the analysis of software processes for effective software development. Section 4 depicts the result analysis in detail. Finally, the conclusion and future work is discussed in Section 5.

2. Related work

This section of article enlightens the major work done in this domain so far. André N. Meyer et al. [3] tried to differentiate between a good workday and a typical workday for a software developer. To identify the answer, the authors devoted a plenty of time to understand working style of software developers. They collected and analyzed approx. 6000 feedbacks from professionals at Microsoft. They further developed a framework to portray the developer's workday from being good and typical. Roopak Sinha et al. [4] presented a survey of discrete formal methods for developing comparatively more trustworthy industrial automation systems. Authors further classified their findings based on SDLC model selection, research gaps and future scope for each phases of SDLC. Ashish Sharma et al. [5] firstly identified the complications in developing the software. Through their study and research, the authors find out that improper documentation of SRS and inadequate estimation is the root cause of the failure of software development. The authors presented a framework in this context to do early estimations of software complexities that are based on software requirement specifications.

Naveed Ali and Richard Lai [6] presented a systematic review on Global Software Development, which is multi-site software development. The authors included over 200 studies for their research work and identified the advantages of software development with reduced cost and having skills professionals. The authors further compared their research findings with previous research work done in the area of GSD for better and quality-enabled software development. Sangeeta Phogat et al. [7] developed a failure rate model that is based on iterative SDLC. After considering discrete requirements at each phase of the development process, the authors incorporated a modulation factor. The authors further validated their model on different iterations of the Eclipse software failure dataset and JDT software failure dataset. Rafeel Parizi et al. [8] presented a study related to the finding usage of the design thinking while software development processes. The authors reviewed over 125 research articles to determine the role of design thinking along with its associated models and approaches. As an outcome, the authors suggested four strategies that can be incorporated in design thinking during software development.

Sorin Nădăban, Simona Dzitac, and Ioan Dzitac [9] surveyed various fuzzy models that are applicable for decision-making approaches. After their rigorous research, the authors mentioned the importance and advantages of the fuzzy TOPSIS method over other methods. Further, the authors also presented some major application areas of the fuzzy TOPSIS method. Krzysztof Palczewski and Wojciech Sałabun [10] presented a

systematic review on various application areas of fuzzy TOPSIS. The authors find the fuzzy TOPSIS method as one of the most useful multi-criteria decision analysis methods. The authors incorporated 25 studies for their review and analysis & they identified that fuzzy TOPSIS is a widely accepted method in different areas of research and development like engineering, health sector, supply chain, environment, etc.

Arooj Adeep et al. [11] developed an m-polar fuzzy linguistic method that is applicable for the multi-criteria decision-making approach. Their approach is adequate to figure out the ideal choice among the all examined alternatives. The authors further presented an algorithm along with the coding. Aamir AA Rahim et al. [12] presented the utilization possibility of Fuzzy TOPSIS MCDM in terms of assessing health, safety, risk factors related to environment etc. Authors after eliminating the complex structure identified that fuzzy TOPSIS is capable enough to identify, examine, and give the rank of material alternatives. Decision makers assigned the initial weights to each alternatives that were further converted into fuzzy values.

3. Analyzing software processes for effective software development

In the era of the customer-centric business world, it is the need of the business to give the highest priority to the customer's expectations. In the development of software products, it is quite glaring that customers can change the requirements of the undergoing software even in the later stages. Sometimes, requirements are not changed intentionally. It may take place due to changes in any government policies, rules or regulations. In such cases, developers of the product have to adjust the software system. These unexpected and irregular changes increase the complexity level of the project. Therefore, there is a need of selecting an appropriate software development life cycle model for the particular software product. The selection of relevant SDLC model is a quite challenging task, which can be performed by professionals and organizations. This selection of SDLC models can either increase or decrease the entire process and complexity level of the project. So one has to precisely analyze all the concerned data related to the project and then finalize the SDLC model.

SDLC is a step by step process which every professional must follow so that a better and quality software can be handed over to the clients [13]. Throughout SDLC, proper planning and analysis of risk & inter-related activities are very crucial. As per the literature [2], most of the organizations do not devote enough time to planning and analysis. It may be hazardous to the software product as software may fail at the end. In this context, it is always advisable to spend sufficient time on each phase of the software development life cycle as this practice can decrease the possibilities of software failure. By investing abundant efforts in analysis and planning can also prevent the project from being over budget and late delivery.

3.1. Study of software development life cycle (SDLC)

Software Development Life Cycle is a systematic approach used to developed high quality software systems that ensure customer expectations, quality and efficiency. The detailed plans for design, implementation, testing, deploying and maintain the software systems are covered in the different phases of SDLC. These phases of SDLC are connected with the next phase as it provides something deliverables and inputs required by the next phase in the life cycle. Gathered requirements are converted into detailed design. The code is implemented as per the design. Testing is performed to verify and validate the requirements. After applying a sufficient amount of testing techniques, the system is ready to be deployed and the system turns into the maintenance phase. Typical phases of SDLC are Requirement elicitation & analysis, Detailed Design, Implementation or coding, Testing, Deployment, and Maintenance respectively.

SDLC methodologies provide the path to navigate through the

process of software development. There are so many recognized SDLC models in use. Every model has its unique features and limitations. In this article, we have chosen the six most popular models to compare their core features by applying fuzzy logic and DSS in the form of MCDM. These models are:

- 1 Spiral Model,
- 2 Agile Software Development,
- 3 Waterfall Model,
- 4 Rapid Application Development (RAD) Model,
- 5 Incremental Model, and
- 6 V-Process Model

3.2. Selection of appropriate software development process using distance measure through fuzzy method

In this section, we applied the Fuzzy TOPSIS method on different criteria of all six SDLC models. The values of these criteria are taken in the form of linguistic terms from five different decision-makers who belong to the different software development industries. These linguistic terms are converted into numeric values by applying a triangular member function of Fuzzy logic. Once the values are converted, we applied Fuzzy TOPSIS method for identifying the appropriate model among all six SDLC models.

TOPSIS and AHP methods were used in the early stages of MCDM. However, for most MCDM scenarios, Fuzzy TOPSIS is the best choice for ensuring decision efficiency. By following this procedure, we can arrive at a decision that is close to the ideal solution. Chen [14] developed the fuzzy TOPSIS method in 2000. The fuzzy Method, which employs triangular fuzzy numbers, extends the TOPSIS approach to a fuzzy environment. The numeric linguistic values are replaced with rating and weighting factors using triangular fuzzy numbers. This method is used to compute multiple alternatives against the predetermined criteria [15, 16]. Fuzzy TOPSIS is used in this article in scenarios where the performance value is in the form of a decision matrix. These are not precise numerical values, but rather linguistic terms provided by the decision-makers.

3.3. Execution of Fuzzy TOPSIS method using triangular membership function

Triangular membership function of fuzzy logic [17, 18] can be applied by using min and max as follows:

$$\text{Triangle}(x; a, b, c) = \max\left(\min\left(\frac{x-a}{b-a}, \frac{c-x}{c-b}\right), 0\right) \quad (1)$$

Three parameters {a, b, c} (where $a < b < c$) determine the x co-ordinates of the three corners of the underlying triangular membership function.

Some important criteria are identified and selected for computation in order to effectively select an ideal framework. These are the criteria that influence the software system's processing and achievement [19, 20]. Our computations are based on the following criteria: scalability, requirement understanding, development cost (money), complexity, and integrity & security. The weight factors for these criteria are set as follows: High, Very High, Average, Average, and Very High. The fuzzy TOPSIS method consists of several steps, beginning with the construction of the matrix of compiling the alternatives versus criteria for the alternatives and ending with the closeness coefficient and ranking of all the alternatives.

To find the best software development process, firstly the alternative ratings from all five-decision makers are gathered. Weights are assigned to selected criteria for each SDLC model as determined by five decision-makers. These criteria were initially weighted in linguistic terms as Very High, High, Average, Low, and Very Low. The widely used triangular membership function of Fuzzy Logic [21] is used in the following steps

as triangular shapes represent fuzzy numbers [22]. According to the triangular membership function, every linguistic term can be assigned with a fuzzy number as very high (7,9,9), high (5,7,9), average (3,5,7), low (1,3,5), and very low (1,1,3). The 5-point scale can be fuzzified in this way. Then, using this 5-point conversion scale, the linguistic terms are replaced with a fuzzy number. We will get five matrices after replacing the values as shown in Table 1 below.

This is the decision matrix from five decision-makers. Now, there is a need to have a single decision matrix that is the combination of all five-decision matrix for calculation purposes. These formulas are used to combine all of the five-decision matrices (equation 2 and equation 3).

$$\tilde{x}_{ij} = (a_{ij}, b_{ij}, c_{ij}) \quad (2)$$

$$a_{ij} = \min_k \{a_{ij}^k\}, b_{ij} = \frac{1}{k} \sum_{k=1}^k b_{ij}^k, c_{ij} = \max_k \{c_{ij}^k\} \quad (3)$$

As there are five decision-makers here, the value of K ranges from 1 to 5. The i value represents the row number, which is the number of alternatives, and the j value represents the number of columns, which is the number of criteria. The values of each cell using the above formulae need to be calculated. Weightage can be converted into a fuzzy number which are currently in linguistic terms as shown in Table 2.

In the next step, the calculation of normalized fuzzy decision matrix will be performed. Computation of normalized fuzzy decision matrix can be identified by applying the following formulae:

$$\tilde{r}_{ij} = \left(\frac{a_{ij}}{c_j^*}, \frac{b_{ij}}{c_j^*}, \frac{c_{ij}}{c_j^*} \right) \text{ and } c_j^* = \max_i \{c_{ij}\} (\text{benefit condition}) \quad (4)$$

$$\tilde{r}_{ij} = \left\{ \frac{a_i^-}{c_{ij}^-}, \frac{a_i^-}{b_{ij}^-}, \frac{a_i^-}{a_{ij}^-} \right\} \text{ and } a_i^- = \min_i \{a_{ij}\} (\text{cost condition}) \quad (5)$$

However, initially, we have to identify the beneficial and non-beneficial criteria. Non-beneficial criteria are also known as cost criteria. scalability, requirement understanding and integrity & security are beneficial criteria. While development cost and complexity are non-beneficial or cost criteria in which minimum value is desired. Now the values of c_j^* for the beneficial criteria & a_i^- for the cost criteria will be determined. Next, the weighted normalized fuzzy decision matrix can be computed by using following formula:

$$\tilde{v}_{ij} = \tilde{r}_{ij} * w_j \quad (6)$$

Below is a formula for multiplying two fuzzy numbers. In multiplication, the components get multiplied to each other that is A component will get multiplied with A component, B component will get multiplied with B component and C component will get multiplied with the C component.

$$\tilde{A}_1 \tilde{A}_2 = (a_1, b_1, c_1) (a_2, b_2, c_2) = (a_1 * a_2, b_1 * b_2, c_1 * c_2) \quad (7)$$

On solving, a weightage normalized fuzzy decision matrix will be obtained. After getting weighted normalized fuzzy decision matrix, it is time to compute the Fuzzy Positive Ideal Solution (FPIS) and Fuzzy Negative Ideal Solution (FNIS). These are the formulae used for calculating FPIS and FNIS.

$$A^* = (\tilde{v}_1^*, \tilde{v}_2^*, \dots, \tilde{v}_n^*), \text{ where } \tilde{v}_j^* = \max_i \{v_{ij}\} \quad (8)$$

$$A^- = (\tilde{v}_1^-, \tilde{v}_2^-, \dots, \tilde{v}_n^-), \text{ where } \tilde{v}_j^- = \min_i \{v_{ij}\} \quad (9)$$

In the formula, it is observed that v_{ij3} that is the c component of the fuzzy number is looked upon for A^* , that is FPIS while the minimum value of the A component that is v_{ij1} that is the A component of the fuzzy number is looked upon for A^- that is FNIS as mentioned in Table 3.

Next, the distance of each alternative from FPIS and FNIS will be determined. Chen [14] introduce a vertex method to calculate the distance between two fuzzy numbers. The following is the formula used to

Table 1

Weight matrix for all alternative models against criteria by all five-decision makers.

Weightage → Criteria →	High Scalability	Very High Requirement Understanding	Average Development Cost	Average Complexity	Very High Integrity & Security
Decision Maker (DM) 1					
Spiral	3,5,7	7,9,9	5,7,9	5,7,9	5,7,9
Agile	7,9,9	3,5,7	1,3,5	1,3,5	5,7,9
Waterfall	5,7,9	5,7,9	3,5,7	3,5,7	3,5,7
RAD	1,3,5	3,5,7	1,1,3	1,1,3	5,7,9
Incremental	3,5,7	5,7,9	1,3,5	3,5,7	5,7,9
V-Process	3,5,7	5,7,9	5,7,9	5,7,9	7,9,9
Decision Maker (DM) 2					
Spiral	5,7,9	7,9,9	5,7,9	5,7,9	7,9,9
Agile	7,9,9	3,5,7	1,1,3	1,3,5	5,7,9
Waterfall	5,7,9	5,7,9	3,5,7	3,5,7	5,7,9
RAD	1,3,5	3,5,7	1,1,3	1,1,3	3,5,7
Incremental	3,5,7	7,9,9	3,5,7	1,3,5	5,7,9
V-Process	3,5,7	7,9,9	5,7,9	5,7,9	5,7,9
Decision Maker (DM) 3					
Spiral	3,5,7	5,7,9	5,7,9	5,7,9	7,9,9
Agile	5,7,9	3,5,7	1,3,5	1,1,3	7,9,9
Waterfall	5,7,9	3,5,7	3,5,7	3,5,7	5,7,9
RAD	1,1,3	1,3,5	1,1,3	1,1,3	3,5,7
Incremental	1,3,5	5,7,9	3,5,7	3,5,7	7,9,9
V-Process	5,7,9	7,9,9	5,7,9	5,7,9	7,9,9
Decision Maker (DM) 4					
Spiral	3,5,7	5,7,9	1,3,5	3,5,7	5,7,9
Agile	5,7,9	5,7,9	3,5,7	1,3,5	7,9,9
Waterfall	3,5,7	5,7,9	5,7,9	7,9,9	5,7,9
RAD	1,3,5	5,7,9	3,5,7	3,5,7	7,9,9
Incremental	5,7,9	7,9,9	5,7,9	5,7,9	7,9,9
V-Process	1,3,5	3,5,7	1,1,3	1,1,3	5,7,9
Decision Maker (DM) 5					
Spiral	5,7,9	5,7,9	5,7,9	3,5,7	5,7,9
Agile	5,7,9	7,9,9	3,5,7	1,3,5	7,9,9
Waterfall	3,5,7	5,7,9	5,7,9	7,9,9	5,7,9
RAD	1,3,5	5,7,9	3,5,7	3,5,7	7,9,9
Incremental	5,7,9	5,7,9	1,3,5	5,7,9	7,9,9
V-Process	1,3,5	5,7,9	1,1,3	3,5,7	5,7,9

Table 2

Decision matrix with corresponding weights.

Weightage → Criteria →	5,7,9 Scalability	7,9,9 Requirement Understanding	3,5,7 Development Cost	3,5,7 Complexity	7,9,9 Integrity & Security
Spiral	3,5,8,9	5,7,8,9	1,6,2,9	3,6,2,9	5,7,8,9
Agile	5,7,8,9	3,6,2,9	1,3,4,7	1,2,6,5	5,8,2,9
Waterfall	3,6,2,9	3,6,6,9	3,5,8,9	3,6,6,9	3,6,6,9
RAD	1,2,6,5	1,5,4,9	1,2,6,7	1,2,6,7	3,7,9
Incremental	1,5,4,9	5,7,8,9	1,4,6,9	1,5,4,9	5,8,2,9
V-Process	1,4,6,9	3,7,4,9	1,4,6,9	1,5,4,9	5,7,8,9

Table 3

Computation of FPIS & FNIS

Weightage → Criteria →	5,7,9 Scalability	7,9,9 Requirement Understanding	3,5,7 Development Cost	3,5,7 Complexity	7,9,9 Integrity & Security
Spiral	1.665,4.508,9	3.885,7.803,9	0.333,0.805,7	0.333,0.805,2.331	3.885,7.803,9
Agile	2.775,6.069,9	2.331,6.201,9	0.429,1.47,7	0.6,1.925,7	3.885,8.199,9
Waterfall	1.665,4.823,9	2.331,6.597,9	0.333,0.86,2.331	0.333,0.755,2.331	2.331,6.597,9
RAD	0.555,2.023,4.995	0.777,5.4,9	0.429,1.925,7	0.429,1.925,7	2.331,6.993,9
Incremental	0.555,4.2,9	3.885,7.803,9	0.333,1.085,7	0.333,0.925,7	3.885,8.199,9
V-Process	0.555,3.577,9	2.331,7.398,9	0.333,1.085,7	0.333,0.925,7	3.885,7.803,9
A*	2.775 6.069,9	3.885,7.803,9	0.429,1.925,7	0.6,1.925,7	3.885,8.199,9
A ⁻	0.555,2.023,4.995	0.777, 5.4, 9	0.333, 0.805,2331	0.333,0.755,2.331	0.755,6.597,9

calculate the distance:

$$d(\tilde{x}, \tilde{y}) = \sqrt{\frac{1}{3}[(a_1 - a_2)^2 + (b_1 - b_2)^2 + (c_1 - c_2)^2]} \quad (10)$$

The distance from each alternative to the FPIS and to the FNIS can be computed using following formula:

$$d_i^* = \sum_{j=1}^n d = (\tilde{v}_{ij}, \tilde{v}_j^*) \quad (11)$$

By applying Eqs. (10) and (11),

Table 4

Computation of the distance from each alternative to the FPIS

Criteria →	Scalability	Requirement Understanding	Development Cost	Complexity	Integrity & Security	d_i^+
Spiral	1.106	0.000	1.868	2.776	0.229	5.979
Agile	0.000	1.289	0.262	0.000	0.000	1.551
Waterfall	0.963	1.136	3.274	2.783	1.288	9.444
RAD	3.528	2.268	0.000	0.099	1.136	7.031
Incremental	1.675	0.000	1.818	0.597	0.000	4.090
V-Process	1.927	0.927	1.818	0.597	0.229	5.498

Table 5

Computation of the distance from each alternative to the FNIS

Criteria →	Scalability	Requirement Understanding	Development Cost	Complexity	Integrity & Security	d_i^-
Spiral	2.796	2.268	2.696	0.913	1.136	9.809
Agile	3.528	1.009	3.238	2.783	1.288	11.846
Waterfall	2.893	1.132	2.020	0.000	0.000	6.045
RAD	0.000	0.000	3.280	3.285	0.229	6.794
Incremental	2.632	2.268	3.219	2.697	1.288	12.104
V-Process	2.480	1.461	3.219	2.697	1.136	10.993

$$d_i^- = \sum_{j=1}^n d = (\tilde{v}_{ij}, \tilde{v}_j^-) \quad (12)$$

The closeness coefficient can be computed by using the following formula.

$$CC_i = d_i^- / d_i^+ + d_i^- \quad (13)$$

On solving, we get the value of CC_i that is the closeness coefficient. Table 6 shows closeness coefficient for all alternative models.

Based on closeness coefficient rank of all the alternatives can be determined. The highest value of closeness coefficient is considered as ranked 1 & the lowest value of closeness coefficient is considered as the last rank alternative. Through Table 7, it is observed that the Agile software development model has the highest value of CC_i so it is ranked as 1 while the Waterfall model has the lowest value of CC_i so it is ranked as 6.

4. Result analysis

This section analyzes the result of applying the Fuzzy TOPSIS based multi-criteria decision-making approach on six alternative models. To identify the suitable software development life cycle model in terms of ranking. In order to determine it, five decision-makers were asked to give their weightage for six alternative models on five different evaluation criteria. Inputs received from decision-makers in the form of linguistic terms are the alternative rating and to identify the relevant weightage to different criteria. By applying the steps of Fuzzy TOPSIS, we identify the closeness coefficients and further ranking for all six models. The obtained closeness coefficient for alternative models can be utilized to represent the degree of inferiority or superiority of the alternative models among each other. However, a detailed result analysis can be represented to focus on the calculated distance from each alternative to FPIS, distance from each alternative to FNIS as well as closeness coefficient for each alternative models.

Table 6

Closeness coefficients for all alternative models

	d_i^+	d_i^-	CC_i
Spiral	5.979	9.809	0.6213
Agile	1.551	11.846	0.8842
Waterfall	9.444	6.045	0.3903
RAD	7.031	6.794	0.4914
Incremental	4.090	12.104	0.7474
V-Process	5.498	10.993	0.6667

Table 7

Ranking of each alternative

	d_i^+	d_i^-	CC_i	Rank
Spiral	5.979	9.809	0.6213	4
Agile	1.551	11.846	0.8842	1
Waterfall	9.444	6.045	0.3903	6
RAD	7.031	6.794	0.4914	5
Incremental	4.090	12.104	0.7474	2
V-Process	5.498	10.993	0.6667	3

4.1. Result analysis of agile software development

In this direction, the following graph in Fig. 1 representing the values of the 1st ranked model that is the Agile software development. Here distance from each the alternative to FPIS is calculated as 1.551, distance from each the alternative to FNIS is obtained as 11.846 and closeness coefficient is calculated as 0.8842

4.2. Result analysis of incremental model

In Fig. 2, the alternative model that is on rank 2, the Incremental model is represented. Here distance from each the alternative to FPIS is calculated as 4.090, distance from each the alternative to FNIS is obtained as 12.104 and closeness coefficient is calculated as 0.7474

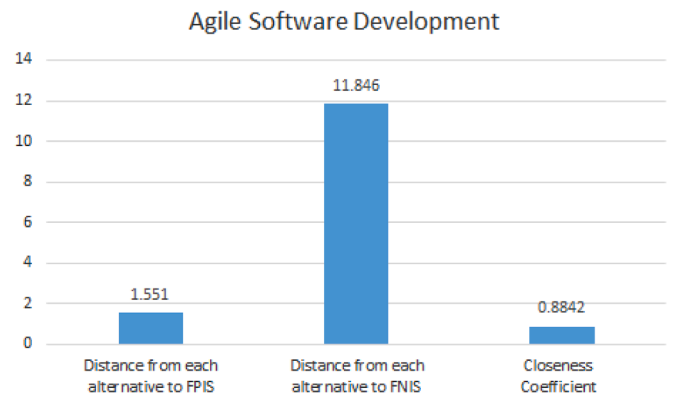


Fig. 1. Distance from each alternative to FPIS, FNIS and closeness coefficient scores for Agile Software Development

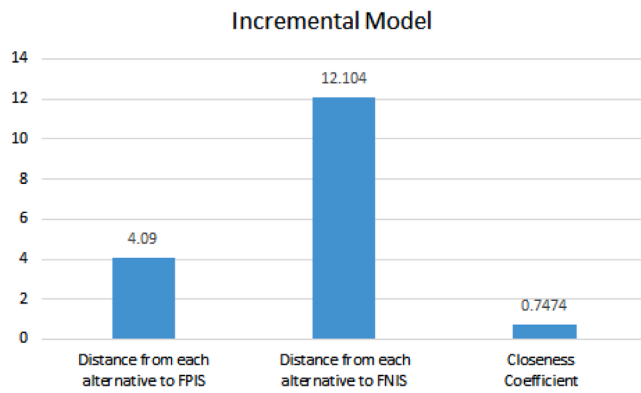


Fig. 2. Distance from each alternative to FPIS, FNIS and closeness coefficient scores for Incremental Model

4.3. Result analysis of V – Process model

Following graph in Fig. 3 representing the values of the 3rd ranked model that is the V-model. Here distance from each the alternative to FPIS is calculated as 5.498, distance from each the alternative to FNIS is obtained as 10.993 and closeness coefficient is calculated as 0.6667

4.4. Result analysis of spiral model

In Fig. 4, the alternative model that is on rank 4, the Spiral model is represented. Here distance from each the alternative to FPIS is calculated as 5.979, distance from each the alternative to FNIS is obtained as 9.809 and closeness coefficient is calculated as 0.6213

4.5. Result analysis of rapid application development model

Following graph in Fig. 5 representing the values of the 5th ranked model that is the RAD model. Here distance from each the alternative to FPIS is calculated as 7.031, distance from each the alternative to FNIS is obtained as 6.794 and closeness coefficient is calculated as 0.4914

4.6. Result analysis of waterfall model

In Fig. 6, the alternative model that is on the last rank that is 6, the Waterfall model is represented. Here distance from each the alternative to FPIS is calculated as 9.444, distance from each the alternative to FNIS is obtained as 6.045 and closeness coefficient is calculated as 0.3903

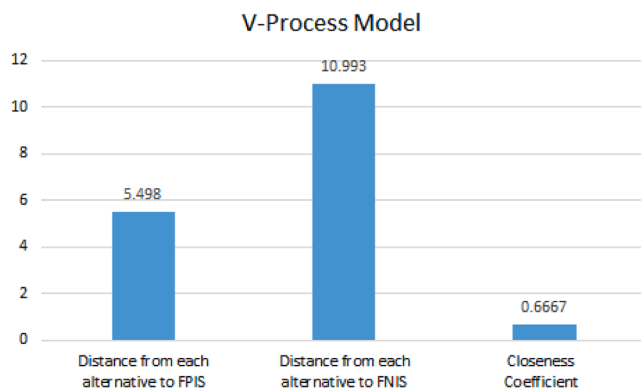


Fig. 3. Distance from each alternative to FPIS, FNIS and closeness coefficient scores for V-Process Model

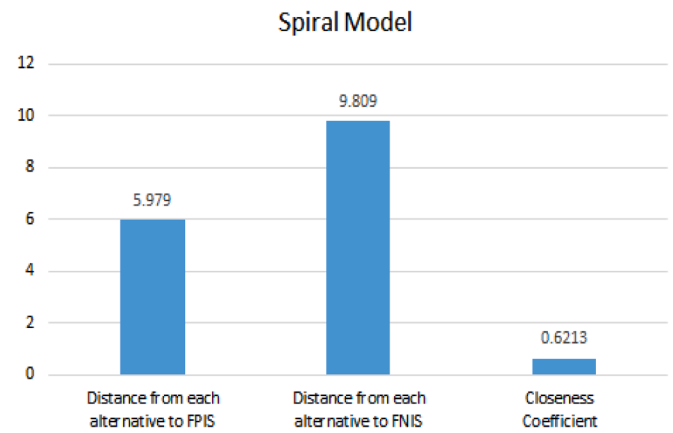


Fig. 4. Distance from each alternative to FPIS, FNIS and closeness coefficient scores for Spiral Model

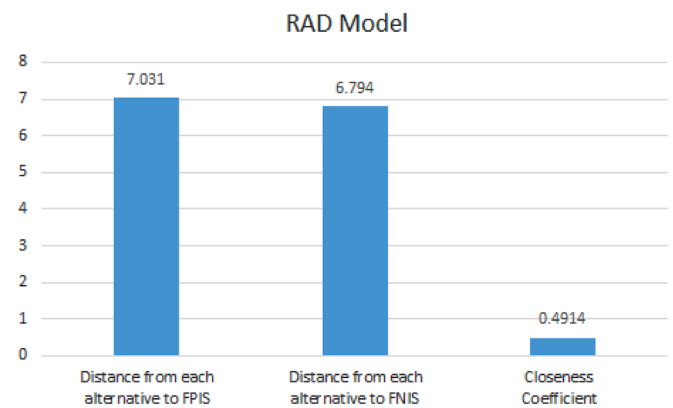


Fig. 5. Distance from each alternative to FPIS, FNIS and closeness coefficient scores for RAD Model

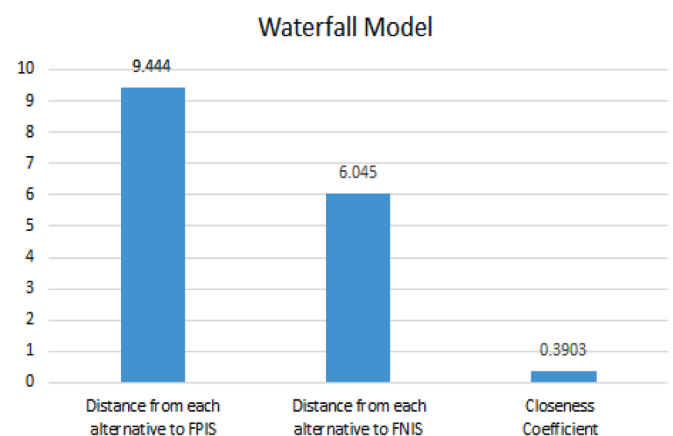


Fig. 6. Distance from each alternative to FPIS, FNIS and closeness coefficient scores for Waterfall Model

4.7. Representation of distance from each alternative to FPIS for all models

Fig. 7 shows the distance from each alternative to FPIS for all alternative models. Here we can observe that the Waterfall model has the highest distance from each the alternative to FPIS as 9.444 and the Agile software development has the lowest value as 1.551

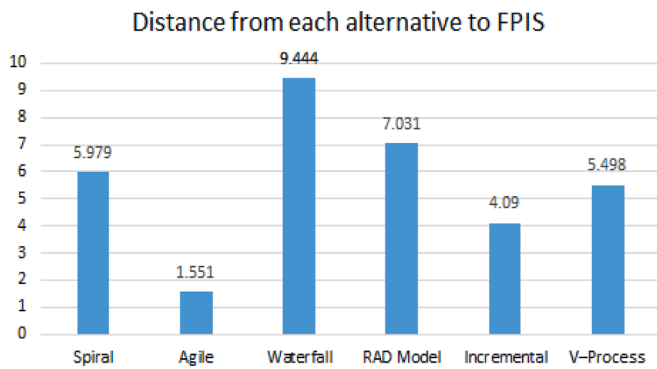


Fig. 7. Distance from each alternative to FPIS (d_i^*) for all alternative models

4.8. Representation of distance from each alternative to FNIS for All models

Similarly, Fig. 8 shows the distance from each alternative to FNIS for all alternative models. Here we can observe that the Incremental model has the highest distance from each the alternative to FNIS as 12.104 and the Waterfall model has the lowest value as 6.045

4.9. Representation of ranking of SDLC models based on closeness coefficient

Fig. 9 shows the closeness coefficients of all alternative models. Based on the closeness coefficient, we can determine the ranking order of all alternative models and select the best one from among a set of feasible alternative models. Here we can observe that the Agile software development has the highest degree of closeness coefficient as 0.8842, so it is ranked as 1. While, the Waterfall model has the lowest degree of closeness coefficient as 0.3903, so it is ranked as 6. Other alternative models are also ranked on the same pattern and we obtained that the Incremental model is ranked as 2nd position, V-Process model is ranked at 3rd position, Spiral model is ranked at 4th position, RAD model is ranked at 5th position and finally, the Waterfall model is ranked at last, 6th position. According to these obtained rank, software professionals will be in a better condition to identify and select the appropriate software development model.

5. Conclusion and future scope

The criteria and their impacts on the selection of relevant SDLC models is a very crucial phase. However, there is not enough information in the literature about how to select an appropriate model for our project. Therefore, there is a need for applying some mathematical decision support systems. A decision support system can help in reducing human effort. It can produce better results if we apply some specialized tools like fuzzy logic and optimization techniques. Fuzzy TOPSIS is one of the MCDM approaches that have the capability to analyze data in linguistic terms by converting it into fuzzy values. It is a simple and precise method to solve multi-criteria decision-making problems. The work presented in the article computes the ranks and closeness coefficients of all the alternative SDLC models based on different criteria. In this article, we collected the corresponding weights of criteria for alternative six SDLC models from five different decision-makers in the form of linguistic terms. These linguistic terms were further converted into fuzzy values. Then Fuzzy TOPSIS technique was applied to get the rank of each and every alternative model. As an outcome, Agile software development is identified as the ideal or most appropriate software development process.

The work contained in this article can be extended with the following points of possible future research issues. In this article, we have considered five constraints as scalability, requirement understanding,

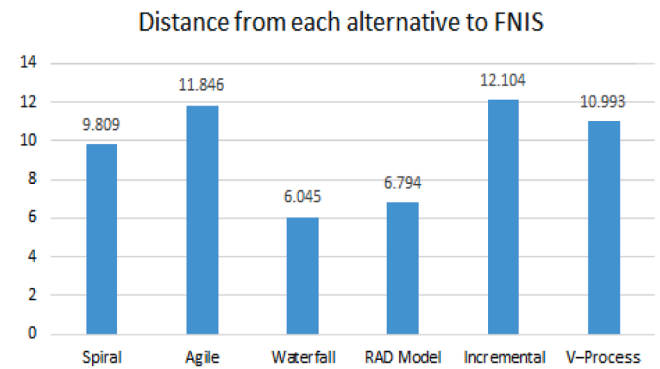


Fig. 8. Distance from each alternative to FNIS (d_i^-) for all alternative models

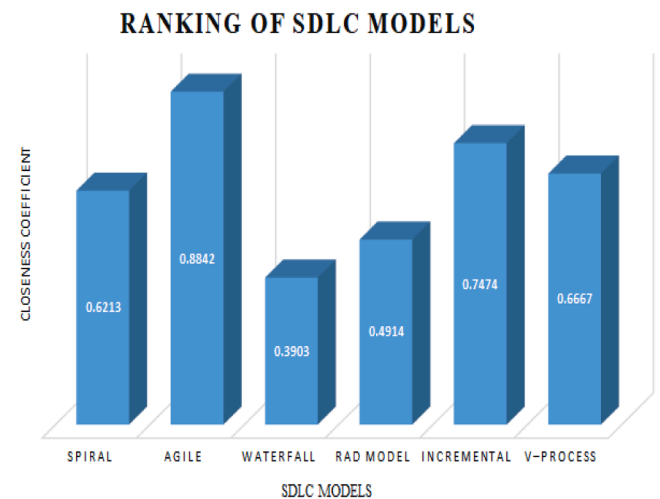


Fig. 9. Ranking of SDLC models based on closeness coefficients

development cost, complexity, and integrity & security. In the future, certain other factors that may affect the estimation of any software project can be added so that estimation can be done more accurately and efficiently. The presented computation in this article is based on six different software development processes. In the future, a machine learning-based tool can be implemented in which we can pass the parameters of any project and a suitable development process can be suggested as per the given constraints and behavior of the project.

CRediT authorship contribution statement

Nikhil Govil: Conceptualization, Writing – review & editing, Methodology, Writing – original draft. **Ashish Sharma:** Supervision, Validation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

- [1] Nayak Veena, D'Souza DrRio. A survey on multi-criteria decision making methods in software engineering. *Int J Innovat Sci Res Technol* July 2018;3(7):366–74.
- [2] Aggarwal KK, Singh Yogesh. *Software Engineering*. Third Ed. New Age International (P) Limited Publishers; 2009.
- [3] Meyer AN, Barr ET, Bird C, Zimmermann T. Today was a good day: the daily life of software developers. *IEEE Trans Softw Eng* 1 May 2021;47(5):863–80. <https://doi.org/10.1109/TSE.2019.2904957>.
- [4] Sinha R, Patil S, Gomes L, Vyatkin V. A survey of static formal methods for building dependable industrial automation systems. *IEEE Trans Ind Inf* July 2019;15(7):3772–83. <https://doi.org/10.1109/TII.2019.2908665>.
- [5] Sharma A, Vardhan M, Kushwaha DS. A Versatile approach for the estimation of software development effort based on SRS document. *Int J Softw Eng Knowledge Eng* 2014;24(01):1–42. <https://doi.org/10.1142/S0218194014500016>.
- [6] Ali N, Lai R. Global software development: a review of its practices. *Malaysian J Comput Sci* 2021;34(1):82–129. Retrieved from, <https://ejournal.um.edu.my/index.php/MJCS/article/view/28581>.
- [7] Sangeeta Sitender, Sharma K, et al. New failure rate model for iterative software development life cycle process. *Autom Softw Eng* 2021;28:9. <https://doi.org/10.1007/s10515-021-00288-9>.
- [8] Parizi Rafael, Prestes Matheus, Marczak Sabrina, Conte Tayana. How has design thinking being used and integrated into software development activities? A systematic mapping. *J Syst Softw* 2022;111217. <https://doi.org/10.1016/j.jss.2022.111217>.
- [9] Nădăban Sorin, Dzitac Simona, Dzitac Ioan. Fuzzy TOPSIS: a general view. *Procedia Comput Sci* 2016;91:823–31. <https://doi.org/10.1016/j.procs.2016.07.088>.
- [10] Palczewski Krzysztof, Saliabun Wojciech. The fuzzy TOPSIS applications in the last decade. *Procedia Comput Sci* 2019;159:2294–303. <https://doi.org/10.1016/j.procs.2019.09.404>.
- [11] Adeel A, Akram M, Koam ANA. Group decision-making based on m-Polar Fuzzy linguistic TOPSIS method. *Symmetry* 2019;11:735. <https://doi.org/10.3390/sym11060735>.
- [12] Rahim AA, Musa SN, Ramesh S, Lim MK. Development of a fuzzy-TOPSIS multi-criteria decision-making model for material selection with the integration of safety, health and environment risk assessment. In: *Proceedings of the Institution of Mechanical Engineers, Part L: Journal of Materials: Design and Applications*. 235; 2021. p. 1532–50. <https://doi.org/10.1177/1464420721994269>.
- [13] Jalote Pankaj. *An integrated approach to software engineering*. Third Ed. Narose Publishing House; 2008.
- [14] Chen CT. Extensions of the TOPSIS for group decision-making under fuzzy environment. *Fuzzy Sets SYST*. Aug 2000;114(1):1–9. [https://doi.org/10.1016/S0165-0114\(97\)00377-1](https://doi.org/10.1016/S0165-0114(97)00377-1).
- [15] Ambrin Rabia, Ibrar Muhammad, De Manuel, Sen La, Rabbi Ihsan, Khan Asghar. Extended TOPSIS method for supplier selection under picture hesitant fuzzy environment using linguistic variables. *Hindawi, J Math* 2021;2021. <https://doi.org/10.1155/2021/6652586>. Article ID 6652586.
- [16] Kore Neelima B, Ravi K, Patil SB. A simplified description of FUZZY TOPSIS method for multi criteria decision making. *Int Res J Eng Technol (IRJET)* May'2017;04(05).
- [17] Sivanandam SN, Deepa SN. *Principles of soft computing*. Second Edition. Wiley India Pvt. Ltd.; 2014.
- [18] Tadesse Admasu, Acharya MM. Manoranjan Sahoo & Srikumar Acharya (2021) Fuzzy linear programming problem with fuzzy decision variables: a geometrical approach. *J Stat Manag Syst* 2022;24(4):853–63. <https://doi.org/10.1080/09720510.2021.1872174>.
- [19] Tiwari A, Lohani QMD, Muhuri PK. Interval-valued intuitionistic Fuzzy TOPSIS method for supplier selection problem. In: 2020 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE); 2020. p. 1–8. <https://doi.org/10.1109/FUZZ48607.2020.9177852>.
- [20] Kanj H, Abi-Char PE. A New Fuzzy-TOPSIS based risk decision making framework for dangerous good transportation. In: 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS); 2019. p. 2666–72. <https://doi.org/10.1109/HPCC/SmartCity/DSS.2019.00374>.
- [21] Zhao Jin, Bose BK. Evaluation of membership functions for fuzzy logic-controlled induction motor drive. In: IEEE 2002 28th Annual Conference of the Industrial Electronics Society. IECON 02. 1; 2002. p. 229–34. <https://doi.org/10.1109/IECON.2002.1187512>.
- [22] Hanoon Wafaa H, Aljassa Safaa M, Fiadh Mahmood S. Nearly 2-absorbing and weakly nearly 2-absorbing fuzzy submodules. *J Discr Math Sci Cryptography* 2020; 23(5):1025–37. <https://doi.org/10.1080/09720529.2020.1714885>.